Home Lab Project Summary

Oct 12, 2026

Johan Erik Villagran · johanvillagran@hotmail.com · github.com/johanVcode

Languages: Swedish (native), English (native level), Spanish (native level)

Overview

I'm Johan, studying to become a DBA, with a strong interest in the infrastructure and security that databases run on. At home I run a lab I built and maintain myself: an OPNsense firewall on a Dell OptiPlex and a Proxmox server made from my old gaming PC. My design goal is systems that run themselves once tuned, with as little room for human error as possible.

Code and projects: github.com/johanVcode

Why this matters for a DBA

A database is only as reliable as the server, storage, network and backups underneath it, and my lab gives me hands-on practice with exactly those layers.

- **Linux servers:** I install, size and harden Debian and Oracle Linux servers myself, starting from minimal templates.
- **Storage:** ZFS, snapshots and growing disks with LVM, the same problems that stop a database when its disk fills up.
- **Security:** deny-by-default network rules, least privilege and no root login, which protect the data as much as the database itself does.
- **Availability:** snapshots today, with a mirrored disk and router failover on the roadmap.
- **Databases:** in class I work with data modeling and Oracle Database in Podman containers on Oracle Linux. Example: database_theory, a data model for a library database.

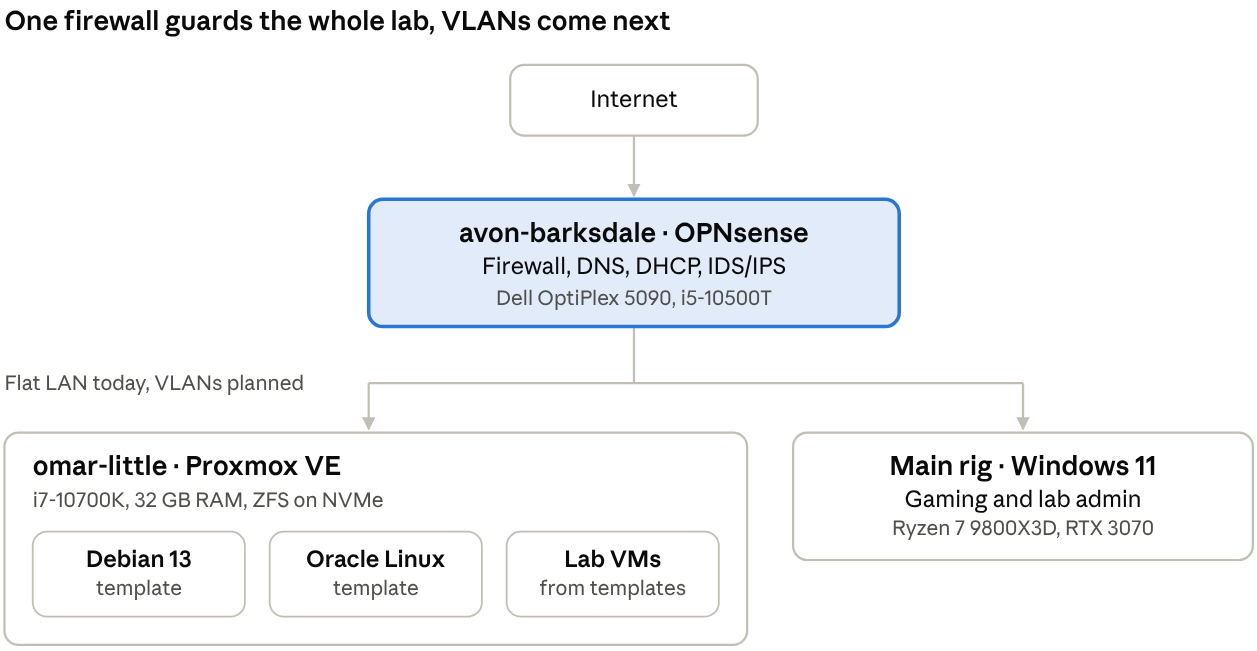
I want to learn from experienced DBAs and grow into the role for the long term.

How I work

I write documentation so that someone with no background can follow it step by step, without guessing.

- **Self-taught by testing.** I learned computers as a kid without manuals: try it, see what breaks, adjust. I still work that way, and I trust what I have tested over what I have only read.
- **Understanding first, terms second.** I learn a concept through an analogy, then attach the real name to it. That is also why I can explain technical things in plain language.
- **Evidence over ego.** When someone shows me facts that prove me wrong, I drop my idea, say I was wrong, and learn from it. I would rather be corrected than stay wrong.
- **AI as a tool, not an answer.** I use AI to help read logs and spot patterns, then verify the result with my own research and testing.
- **Everything documented.** Every project gets its reasons, its limits and its next steps written down. Example: the README for dlorg, my Downloads sorter, covers install, updates, known limitations and how to stop it.

Lab at a glance



home lab layout · firewall, hypervisor, admin PC

Everything sits behind the OPNsense firewall; the Proxmox host runs my Linux VMs, and the main rig is where I manage it all.

Firewall design

The firewall blocks by default. I disabled the built-in "allow LAN to anything" rules and gave every device group explicit block and allow rules, with all blocks logged.

Group	Planned VLAN	Addresses	Role
Infra	100	.1	The firewall itself
Hypervisor	200	.2–.10	Proxmox hosts
Admin	300	.11–.20	My admin devices
Homelab	400	.21–.40	Lab VMs and services
LAN	500	.41–.60	Everyday computers
Mobile	600	.61–.80	Phones and tablets
IoT	700	.81–.100	Smart devices
Rest	800	.101–.200	Anything unknown

Rule order:

1. Admin devices get full access.
2. Every device may reach the firewall for DNS only.
3. Each group is blocked from the more trusted groups. Mobile, IoT and Rest are also blocked from each other and within their own group.
4. Each group may then reach the internet.

Around the rules: Unbound DNS with DNSSEC and DNS-over-TLS upstreams (Mullvad, Quad9, Cloudflare), a traffic shaper (fq_codel) against lag under load, the web GUI on LAN only over HTTPS, root login disabled and a 15-minute auto-logout.

IDS/IPS design

I split the intrusion rules by how much I trust them: rulesets with few false positives drop traffic automatically, broad ones only raise an alert for me to review.

Policy	Action	Rulesets (examples)
Confirmed malicious	Drop	abuse.ch feeds (Feodo, SSL blacklist, ThreatFox, URLhaus), botnet C2, exploits, phishing, shellcode, worms
Behavioral and broad	Alert only	Scans, malware, coin miners, mobile malware, Tor, suspicious user agents

Rule updates run nightly. IDS/IPS is switched off on the bare-metal firewall for now; I'm tuning it in a test VM first (see Troubleshooting).

Troubleshooting: the Intel I226-V

Moving OPNsense from a VM to bare metal exposed a FreeBSD driver problem with my Intel I226-V network card, which I traced from the logs instead of guessing.

1. I tested OPNsense as a VM on Proxmox first. Linux handled the card there, OPNsense only saw virtual bridges (vmbr0, vmbr1, vmbr2), and everything worked.
2. I moved OPNsense to bare metal, where FreeBSD drives the card directly, and the problem appeared.
3. I used AI to help read the logs, then confirmed through my own research that it is a known FreeBSD issue with this card.

Option	Upside	Downside
Virtualize on a Linux host	Proven to work in my tests	The firewall depends on the host, and the host must have no IP on the WAN bridge
Stay bare metal and tune the card (e.g. turn off Energy Efficient Ethernet)	Simpler, smaller attack surface	Root cause still to verify

Current status: OPNsense runs on bare metal with IDS/IPS turned off. Before switching it back on, I'm tuning it in a headless test VM, so nothing untested goes into production. That work is paused while school comes first.

Scripting: dlorg

dlorg is a Bash script that sorts new files in ~/Downloads into folders by file type, running in the background as a systemd user service.

- Watches the folder with inotifywait and sorts a file once it has been fully written (close_write) or moved in (moved_to).
- Sorts by extension into nine folders (Pictures, Videos, Documents and more); anything unknown goes to Other.
- Renames instead of overwriting when names collide: cat.jpg becomes cat_1.jpg.
- Installed read-only (chmod 544) as a user service, so it never needs root.
- Documented with install, update and stop steps, known limitations, and dev notes explaining each decision.

Efficiency and documentation

I size hardware to the job, weighing power draw against the compute I actually need.

- The firewall runs an Intel i5-10500T, the low-power "T" variant, because routing and filtering don't need a full desktop chip.
- The Proxmox host uses the CPU's integrated graphics, which frees the GTX 1660 Super for a VM.
- Next: two identical routers running as a high-availability (HA) pair with CARP, so one takes over within seconds if the other fails. I'm weighing the Intel N150 against the N350: the N150 draws the least power, while the N350 has twice the cores (8 vs 4) for more IDS/IPS headroom at a similar low power level. My research says the N150 should be enough, so the N350 is my fallback.

I document everything so someone with no background can follow it: the hardware inventory, the network layout, and the reason behind each choice.

Known limitations and roadmap

The network is flat today, so the group rules control traffic to the firewall and the internet but not between devices. That is a known, deliberate stage: the groups are already named for their VLANs, so the switch upgrade is a move, not a redesign.

1. Give every known device a fixed address in its group, and limit the dynamic pool to the Rest range.

2. Switch SSH to key-only login and force all DNS through the firewall.
3. Add WireGuard for remote access, with its own subnet and rules.
4. Buy a managed switch with VLAN and 802.1X support, and move each group onto its VLAN.
5. Run FreeRADIUS on a Raspberry Pi with certificate login (EAP-TLS), so devices land in their VLAN automatically.
6. Re-enable IDS/IPS, and add a second NVMe for a ZFS mirror on the Proxmox host.
7. Add a 5G line as automatic backup when the fiber goes down (multi-WAN), with a rule that blocks heavy traffic to save mobile data.

What this shows

- Network security design: deny by default, least privilege, defense in depth
- Firewall and IDS/IPS administration on OPNsense
- Virtualization with Proxmox and ZFS
- Linux (Debian, Oracle Linux) and FreeBSD
- Troubleshooting from logs, and documenting trade-offs honestly